Язык программирования Python

Основы

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Инструкции и выражения

Инструкция – элемент языка, определяющий действие (команду), которое требуется выполнить.

Конец строки является окончанием инструкции, никаких знаков на конце не требуется.

Исключением являются вложенные инструкции, с помощью которых вы делаете более подробное описание команд.

Инструкции состоят из выражений. Выражения создают и обрабатывают объекты (числа, строки и т.д.).

Выражение – это фрагмент кода, который в итоге даёт какой-то результат (объект с каким-то значением).

Инструкции представляют собой более крупные логические блоки программы – они напрямую используют выражения для обработки данных.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример инструкций

```
>>> print('Первый!')
Первый!
```

Функция print() позволяет нам выводить различную информацию на экран.

```
>>> var = 'Вторая строка'
>>> print(var)
Вторая строка
```

```
>>> var = 2 + 9
>>> print(var)
11
```

Вся строка является инструкцией, а «2 + 9» – выражением.

Инструкции `var = 'Вторая строка'` и `var = 2 + 9` задают значение **переменной**.

Переменная в этих случаях носит имя «var».

Переменные используются для хранения и передачи информации по ходу выполнения программы.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Переменные и литералы

На предыдущем слайде для демонстрации использовались конкретные значения `'Первый!'`, `'Вторая строка'`, `2` и `9`.

Такие **фиксированные значения определённого типа** называют **литералами**.

Для того, чтобы свободно оперировать этими значениями и получать на их основе новые, им присваивают имена.

Таким образом создаются **переменные – имена, которые ссылаются на конкретные значения** (одно за раз) **различных типов данных**. При этом переменные могут менять своё значение в течение работы программы.

Переменная var сначала ссылалась на значение `'Вторая строка'`, а потом на число `11`.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Имена переменных и ключевые слова

Имена переменных задаёт программист, однако есть некоторые **ограничения**:

1. Имя не может начинаться с цифры.
2. Имя не может начинаться с символа операции. Допустимо только нижнее подчёркивание.
3. В качестве имени нельзя использовать ключевое слово.

Также крайне желательно давать переменным «говорящие» имена, исходя из которых можно понять смысл хранимого значения.

Ключевые слова Python – это слова, которые имеют определённый одинаковый смысл во всех программах.

`and`	`as`	`assert`	`break`	`class`	`continue`
`def`	`del`	`elif`	`else`	`except`	`exec`
`finally`	`for`	`from`	`global`	`if`	`import`
`in`	`is`	`lambda`	`nonlocal`	`not`	`or`
`pass`	`raise`	`return`	`try`	`while`	`with`
`yield`	`True`	`False`	`None`	-	-

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Операция присваивания

Для того, чтобы присвоить переменной какое-либо значение, используется операция присваивания, обозначаемая символом «=».

Общая форма записи этой операции:

переменная = выражение

При этом переменной, имя которой указано слева, ссылается на значение, указанное справа. Также этот процесс называется **инициализацией** переменной.

Можно проводить **групповые присваивания**. В этом случае слева от операции присваивания через запятую прописываются переменные, а справа также через запятую соответствующие им значения:

переменная_1, переменная_2,… = выражение_1, выражение_2,…

Выражением может быть комбинация элементов данных (литералов, переменных), знаков операций, функций и т.д., которые в конце приводят к одному значению.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Объявление переменных

```
1  >>> # После символа «решётка» следует комментарий
2  ... a = 30
3  >>> print(a / 3) # Выводим значение переменной a, делённое на 3
4  10.0
5  >>> # Значение может быть присвоено нескольким переменным
6  ... # одновременно. При этом старое значение уже используемых
7  ... # переменных будет переписано и все переменные будут
8  ... # ссылаться на один и тот же объект.
9  ... a = b = 0
10 >>> print(a, b)
11 0 0
12 >>> # Можно проводить групповые присваивания. Переменные и
13 ... # соответствующие им значения записываются через запятую.
14 ... c, d = 5, 12
15 >>> print(c, d)
16 5 12
17 >>> # Переменная должна быть определена перед использованием
18 ... print(e)
19 Traceback (most recent call last):
20     File "<stdin>", line 1, in <module>
21 NameError: name 'e' is not defined
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Семантика и синтаксис

Семантика – система правил истолкования отдельных языковых конструкций. Семантика определяет смысловое значение предложений языка программирования.

Синтаксис – набор правил построения фраз языка программирования, позволяющий определить осмысленные предложения в этом языке.

Алгоритм, реализующий решение задачи, всегда работает с данными.

Данные – это любая информация, представленная в формализованном виде и пригодная для обработки алгоритмом.

Данные, известные перед выполнением алгоритма, являются начальными, **исходными данными**. Данные, используемые в процессе выполнения программы, являются **промежуточными данными**. Результат решения задачи – конечные, **выходные данные**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Типы данных

Одна из основных особенностей любого языка программирования – это способность представлять элементы данных таким образом, чтобы наиболее **точно хранить их значения**, а также **обрабатывать их только с помощью допустимых действий**. За это отвечают типы данных (каждый имеет уникальное название).

Существует несколько видов типов данных:

- **встроенные** – те типы, которые встроены в интерпретатор;
- **не встроенные** – типы данных, которые можно импортировать из библиотек или создавать самим.

Рассмотрим некоторые встроенные типы данных.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

NoneType – отсутствие значения

NoneType – специальный тип, представляющий отсутствие значения.

Имеет только одно значение:

- **None** – неопределенное значение переменной.

Часто используется в качестве значения по умолчанию, а также для проверки корректности чтения из файла, таблицы, базы данных и т.д.

Создать переменную со значением None можно так:

```
null_var = None
```

Пример использования будет рассмотрен вместе с функциями и при создании своих типов данных (классов).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

bool – логический тип

Логический тип данных (**bool**) имеет всего два значения:

- **True** – истина (может быть представлено 1);
- **False** – ложь (может быть представлено 0).

Булевый тип в основном используется в условиях, когда результатом может быть только «да» или «нет».

Создать булевую переменную можно также, как и остальные:

```
tr = True
ng = False
```

Однако чаще всего используются операции сравнения:

```
var = a < 5
```

Пример использования будет рассмотрен в лекции с условными операциями.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Числовые типы

int – целое число. Используются для стандартных арифметических операций, когда нас не интересует точность после запятой.

Целые числа в Python не имеют ограничений по количеству цифр:

…, -5968469568420452 1, …, -1, 0, 1, 2, …, 3216458794658454, …

float – числа с плавающей точкой (или вещественные числа с дробной частью).

Поддерживают те же операции, что и целые, однако из-за компьютерного представления могут возникать неточности.

1.1, 3.14, 2.71 и т.д.

complex – комплексные числа.

Включают действительную и мнимую части:

2+8j, -5.7+7.5j, -8j и т.д.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

str – строковый тип

Строки (**str**) – это упорядоченные последовательности символов, предназначенные для работы с текстовой информацией.

Чтобы присвоить переменной строковое значение, достаточно приравнять выражение в одинарных или двойных кавычках:

```
s1 = 'ain"t it fun?'
s2 = "ain't it fun?"
```

Разницы между одинарными и двойными кавычками нет – они введены для того, чтобы можно было использовать кавычки другого вида внутри предложения, как показано на примере выше.

То есть если записать так:

```
s1 ='ain't it fun?'
```

то интерпретатор выдаст сообщение об ошибке.

Подробнее строки рассмотрим в отдельной лекции.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Последовательности

Последовательности способны **хранить наборы объектов** других типов **упорядоченно** и предоставлять **доступ к ним по порядковому номеру**.

tuple – кортеж. Позволяет хранить набор объектов разного типа (в том числе другие последовательности):

(1, 2.13), ('Иван', 24, 'Дарья', 18), (87, ('Hello', 3)) и т.д.

list – список. Позволяет хранить набор объектов разного типа (в том числе другие последовательности):

[1, 2.13], ['Иван', 24, 'Дарья', 18], [87, ['Hello', 3]] и т.д.

Кортеж является **неизменяемым** типом, а список – **изменяемым**.

Неизменяемость объекта говорит о том, что его значение нельзя изменить. Если необходимо это сделать, то создаётся новый объект с нужным значением. У изменяемых объектов значения могут меняться во время выполнения программы.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Другие коллекции

Например, множества и словари, которые **хранят наборы объектов** других типов **неупорядоченно** (т.е. порядок хранения может не совпадать с порядком заполнения). При этом подразумевается, что **у множества** будет только **проверяться наличие** того или иного значения с помощью ключевого слова **in**, а **доступ к элементам словаря** осуществляется по **заданным ключам**.

set – множества. Изменяемая коллекция, которая хранит наборы неизменяемых объектов. При этом каждое значение будет уникальным (дубликаты удаляются), а сами объекты хранятся неупорядоченно.

{'Hello', 'hell', '42', 5, 9.1}

dict – словари. Неупорядоченная изменяемая коллекция, хранящая пары «ключ-значение». Ключом может быть неизменяемый объект, а значением – любой объект.

{'ID' : 1948, 'Название' : 'Телевизор', 'Диагональ' : 51.5}

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Изменяемые и неизменяемые типы данных

В Python существуют изменяемые и неизменяемые типы.

- Неизменяемые (immutable)
  - Логический тип (bool)
  - Целые числа (int)
  - Числа с плавающей точкой (float)
  - Строки (str)
  - Кортежи (tuple)
- Изменяемые (mutable)
  - Списки (list)
  - Множества (set)
  - Словари (dict)

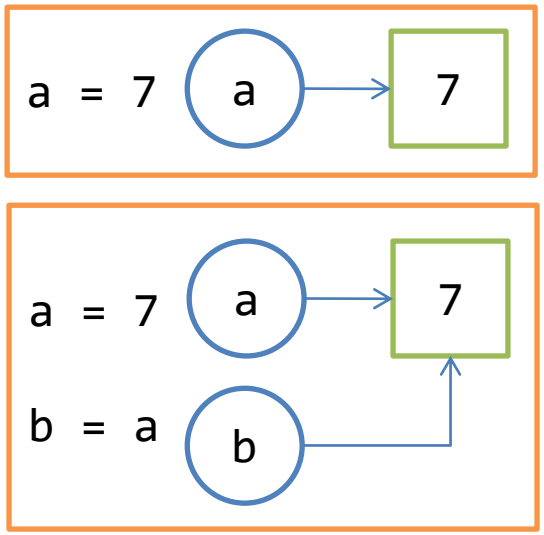
Неизменяемость типа данных означает, что созданный объект больше не изменяется. При необходимости внести изменения создаётся новый объект.

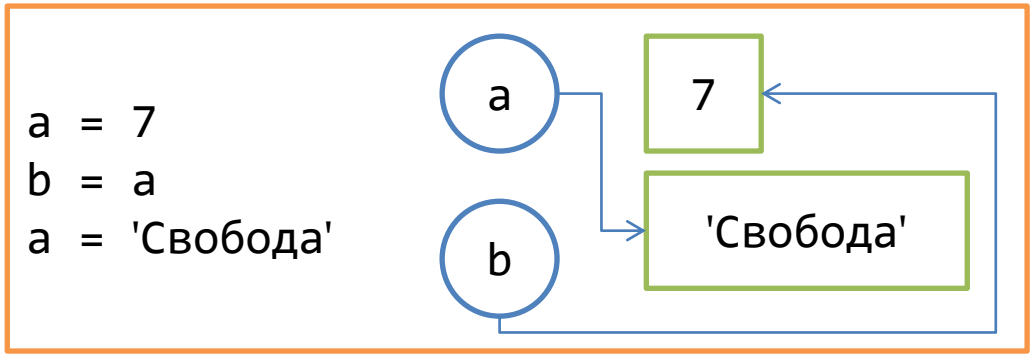
© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Объекты и ссылающиеся на них переменные

На рисунке ниже схематично изображены взаимоотношения между объектами и переменными.

Здесь **кругами обозначены переменные** (ссылки на объекты), а **прямоугольниками – объекты в памяти** и их значения.





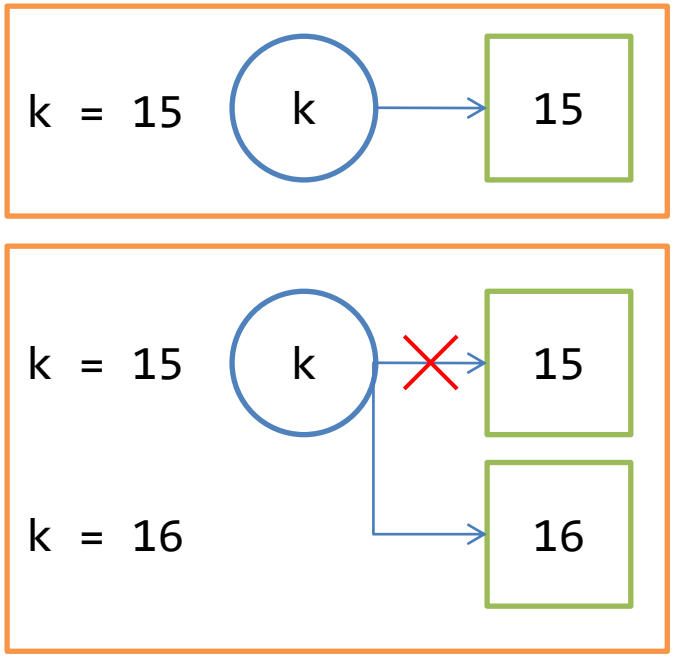
Когда мы записываем код **a = 7**, то создаётся объект типа целое число со значением **7**, а также переменная **a**, в которую записывается ссылка на этот объект.

При записи **b = a** происходит копирование ссылки из переменной **a** в переменную **b** (новый объект не создаётся).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Иллюстрация неизменяемых объектов

Если мы объявим переменную k = 15, то будет создан объект со значением 15 типа int и идентификатором, который можно узнать с помощью функции id():




```
>>> k = 15
>>> id(k)   # узнаём идентификатор
140703713826048
>>> type(k) # узнаём тип
<class 'int'>
>>> k       # узнаём значение
15

>>> k = 16
>>> id(k)   # идентификатор изменился
140703713826080
```

Объект с id = 140703713826048 будет иметь значение 15 и изменить его уже нельзя.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

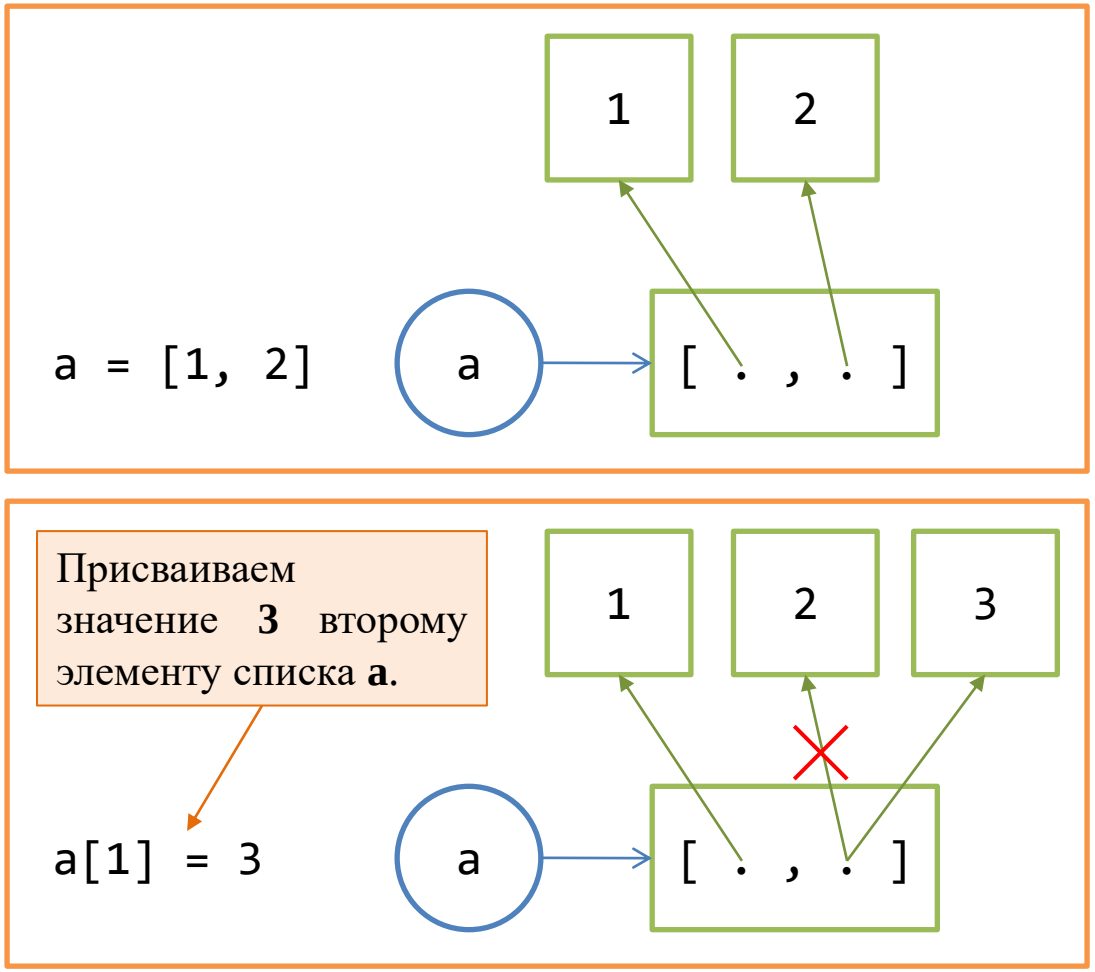
Иллюстрация изменяемых объектов

Если тип данных изменяемый, то можно менять значение объекта.

Например, создадим список [1, 2], а потом заменим второй элемент на число 3:

```
>>> a = [1, 2]
>>> id(a)
2120411614024
>>> type(a)
<class 'list'>
>>> a
[1, 2]

>>> a[1] = 3
>>> id(a)
2120411614024
>>> a
[1, 3]
```



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева